Unidade Curricular de Estruturas de Dados e Algoritmos

1º ano da Licenciatura Ciência de Dados

Projeto da Assiduidade dos Estudantes

2º Semestre

Nº130 662

faags@iscte-iul.pt

Fábio Alexandre Alves Gonçalves

Docente: Profª Maria Pinto Albuquerque

Data de Entrega: 29 / 03 / 2025

Ano Letivo 2024 / 2025

Índice

Introdução..... 3

Limpeza dos dados..... 4

“Justificação da escolha dos 2 algoritmos de ordenação que implementou. Porquê estes e não outros.”.....4

“Resultados, análise do(s) teste(s) de complexidade temporal dos 2 algoritmos de ordenação implementados.”.....5

“Justificação da escolha sobre que TAD (Pilha ou Fila) usar para guardar os dados de assiduidade dos alunos ordenados.”.....6

“Justificação da escolha sobre que representação usou para a implementação do TAD que escolheu (quer tenha escolhido Pilha ou Fila) para guardar os dados de assiduidade ordenados.”.....7

“Quais são as vantagens e desvantagens de utilizar uma pilha em vez de uma fila para armazenar os dados referentes à assiduidade dos alunos?”.....7

“Quem são os alunos que contam com o maior e o menor número de presenças?”.....8

Anexos.....9

Introdução

A gestão académica assume um papel fundamental no acompanhamento do percurso dos estudantes, sendo a gestão da assiduidade um dos seus pilares centrais. A relevância deste processo reside na sua capacidade de monitorizar a presença dos alunos nas atividades letivas, permitindo detetar padrões de frequência, identificar situações de risco (como faltas excessivas) e, consequentemente, promover uma atuação pedagógica mais eficaz e direcionada.

No contexto atual das instituições de ensino, em que a assiduidade é frequentemente um critério obrigatório para aprovação, torna-se essencial garantir o seu controlo rigoroso. Além disso, a assiduidade está diretamente ligada ao sucesso académico, funcionando como um indicador indireto do rendimento e envolvimento dos alunos. Este facto evidencia a importância de uma gestão estruturada e fiável desses dados.

No entanto, com o crescente aumento de alunos e turmas, a gestão manual da assiduidade revela-se pouco eficiente e propensa a erros. Surge assim a necessidade de aplicar métodos computacionais e estruturas de dados adequadas, que possibilitem não só armazenar e aceder rapidamente às informações, mas também automatizar processos e facilitar a análise. Este trabalho insere-se neste âmbito, explorando a utilização de algoritmos de ordenação e estruturas como pilhas, filas e dicionários para otimizar o tratamento e análise dos registos de assiduidade, contribuindo para uma gestão académica mais eficaz, rigorosa e moderna.

Limpeza dos dados

Durante a preparação dos dados de presenças dos alunos, identifiquei 392 registos divergentes, nos quais o mesmo aluno aparece simultaneamente como presente e ausente no mesmo dia. Este tipo de inconsistência levanta questões sobre a qualidade dos dados e, em muitos casos, poderia justificar a sua remoção no processo de limpeza.

No entanto, após uma análise minuciosa, decidi manter esses registos no conjunto de dados final. Esta escolha baseia-se em três fatores principais:

- O enunciado do projeto não exige a eliminação destas inconsistências
- A verificação manual no Excel não revelou erros de preenchimento nem dados em falta
- O enunciado não especifica o número de aulas por dia, deixando em aberto a possibilidade de múltiplos registos legítimos para o mesmo aluno e dia, com presenças e ausências em momentos distintos.

Enquanto estudante da unidade curricular de Ciência de Dados, reconheço a importância de aplicar técnicas de tratamento de dados com responsabilidade. Remover dados sem uma justificação clara pode comprometer a integridade e a profundidade da análise. Assim, a decisão de manter estes registos visa preservar a transparência do processo e permitir análises futuras mais detalhadas sobre a origem e o impacto destes conflitos.

---

1) **"Justificação da escolha dos 2 algoritmos de ordenação que implementou. Porquê estes e não outros."**

Neste projeto foram estudados cinco algoritmos de ordenação, Bubble Sort, Insertion Sort, Selection Sort, Merge Sort e Quick Sort, com o propósito de organizar os registos de assiduidade dos alunos. A escolha final centrou-se no Insertion Sort e no Merge Sort, tanto pelas suas propriedades técnicas como pela possibilidade de contrastar dois algoritmos com comportamentos distintos em termos de eficiência e aplicação prática.

O ficheiro analisado contém 1006 registos, um volume que, apesar de não ser muito elevado, é suficiente para justificar a comparação de desempenho entre algoritmos. O Insertion Sort mostrou-se apropriado para ficheiros de dimensão reduzida ou com dados já parcialmente ordenados, o que pode acontecer quando as presenças são registadas de forma cronológica. Por outro lado, o Merge Sort foi selecionado pela sua capacidade de manter um bom desempenho mesmo em listas de maior dimensão ou com os dados desorganizados, além de preservar a ordem original dos registos com chaves iguais. A escolha destes dois algoritmos permite observar e comparar dois contextos de ordenação distintos: um mais simples e direto, outro mais completo e escalável.

O Bubble Sort e o Selection Sort foram excluídos devido à sua complexidade $O(n^2)$, o que os torna pouco eficazes em listas com muitos elementos ou com ordem aleatória.

O Insertion Sort revela-se eficiente quando a lista já se encontra quase ordenada, podendo funcionar em tempo próximo de O(n), apesar de, no pior cenário, ter uma complexidade de $O(n^2)$.

O Merge Sort, por sua vez, assegura um desempenho consistente com complexidade O(n log n) em todos os casos. Sendo um algoritmo estável, garante a manutenção da ordem relativa dos dados com valores repetidos. A sua principal desvantagem é o consumo extra de memória necessário durante o processo de ordenação.

O Quick Sort, embora seja também recursivo e habitualmente rápido, foi descartado por não garantir estabilidade e poder apresentar complexidade $O(n^2)$ em situações desfavoráveis, o que compromete a consistência e previsibilidade do seu desempenho.

Desta forma, a inclusão do Insertion Sort e do Merge Sort permite analisar abordagens diferentes à ordenação de dados: uma solução iterativa, simples e eficaz para pequenos volumes, e outra recursiva, mais adequada a listas de maior dimensão. Esta comparação reforça a compreensão das vantagens e limitações de cada método em cenários reais de aplicação.

---

2) "**Resultados, análise do(s) teste(s) de complexidade temporal dos 2 algoritmos de ordenação implementados.**"

A comparação entre os resultados obtidos nos dois cenários – com e sem limpeza de dados – permite perceber com clareza o impacto da qualidade dos dados no desempenho dos algoritmos Insertion Sort e Merge Sort. Esta análise é reforçada pelos gráficos de barras gerados para cada conjunto de testes, que representam visualmente a diferença nos tempos de execução.

Nos testes com 50 registos, tanto nos dados limpos como nos dados originais, os tempos de execução de ambos os algoritmos são muito próximos. Nos gráficos correspondentes, as barras de Insertion Sort e Merge Sort aparecem com alturas semelhantes e muito reduzidas, o que confirma que, em listas pequenas, a organização dos dados tem um impacto quase nulo. Este cenário evidencia que o custo computacional de ambos os algoritmos é praticamente irrelevante quando o volume de dados é mínimo, independentemente da presença de registos conflituantes.

No caso dos 500 registos, os gráficos começam a revelar diferenças mais significativas. A barra do Insertion Sort cresce de forma notável, especialmente no gráfico referente ao cenário sem limpeza. Esta diferença entre os dois gráficos mostra que a presença de dados desorganizados e contraditórios afeta negativamente o desempenho do Insertion Sort, levando a um aumento considerável do tempo de execução. Por outro lado, a barra do Merge Sort mantém-se baixa e semelhante nos dois gráficos, o que confirma a sua robustez e estabilidade, mesmo perante dados com inconsistências.

A discrepância torna-se ainda mais evidente quando observamos os gráficos relativos ao cenário com todos os registos disponíveis. No gráfico referente aos dados sem limpeza, a barra do Insertion Sort é visivelmente a mais alta de todos os testes realizados, ultrapassando os 0,09 segundos. Esta diferença drástica em relação ao Merge Sort é visualmente impactante e traduz de forma clara a fragilidade do Insertion Sort face a grandes volumes de dados desorganizados. No gráfico correspondente aos dados limpos, a barra do Insertion Sort é substancialmente mais baixa (cerca de 0,0036 s), o que evidencia como a remoção de registos conflituantes melhora consideravelmente a sua eficiência.

Em contraste, a altura da barra do Merge Sort nos dois últimos gráficos praticamente não varia. A sua consistência gráfica reforça a sua eficiência algorítmica estável, apoiada na complexidade O(n log n), e a sua menor sensibilidade à desordem dos dados. Esta estabilidade torna-o uma opção mais adequada quando se lida com volumes maiores de dados ou com ficheiros de qualidade variável.

Os gráficos de barras desempenham aqui um papel fundamental, ao permitirem uma interpretação imediata e visualmente clara do impacto da limpeza de dados. Através das diferenças de altura entre as barras, é possível verificar que o Insertion Sort é altamente dependente da organização dos dados, enquanto o Merge Sort também se manteve eficaz independentemente do cenário.

---

3) **"Justificação da escolha sobre que TAD (Pilha ou Fila) usar para guardar os dados de assiduidade dos alunos ordenados."**

Para armazenar os dados de assiduidade dos alunos de forma ordenada, foi escolhida a estrutura Fila (Queue), tendo em conta a necessidade de manter a sequência cronológica dos registos. A análise da assiduidade exige que os dados sejam processados na ordem em que foram registados, o que é compatível com o princípio FIFO (First In, First Out), seguido pela Fila.

Durante a preparação dos dados, estes foram ordenados por data, reforçando a importância de preservar essa ordem ao longo do processo de armazenamento e leitura. A estrutura Fila permite manter essa coerência temporal, essencial para uma análise correta da evolução da assiduidade ao longo do tempo.

Foram também realizados testes de desempenho para avaliar o tempo de armazenamento com diferentes estruturas. A Fila registou um tempo de inserção de aproximadamente 0.000148 segundos, enquanto a Pilha apresentou um tempo ligeiramente inferior, cerca de 0.000118 segundos. Apesar desta pequena diferença, a Fila foi escolhida por cumprir, de forma mais adequada, os requisitos lógicos e funcionais do projeto.

---

4) **“Justificação da escolha sobre que representação usou para a implementação do TAD que escolheu (quer tenha escolhido Pilha ou Fila) para guardar os dados de assiduidade ordenados.”**

A implementação da Fila foi realizada com recurso à estrutura lista (*list*) do Python, pela sua simplicidade e adequação aos objetivos do projeto. A inserção dos elementos foi feita com o método append(), que adiciona os registos no final da lista, e a remoção com pop(0), que retira o primeiro elemento, reproduzindo o comportamento FIFO necessário à Fila.

Apesar de o método pop(0) ter complexidade linear, o número de registos utilizados foi suficientemente reduzido para que tal não afetasse significativamente o desempenho do programa. Os testes realizados confirmaram a viabilidade da abordagem.

Assim, a lista mostrou-se uma solução eficaz para representar a Fila, assegurando a preservação da ordem dos registos e permitindo uma implementação clara e objetiva.

---

5) “**Quais são as vantagens e desvantagens de utilizar uma pilha em vez de uma fila para armazenar os dados referentes à assiduidade dos alunos?”**

A estrutura Pilha (Stack), que opera segundo o princípio LIFO (Last In, First Out), apresenta algumas vantagens técnicas em relação à Fila (Queue), sobretudo no que diz respeito ao desempenho. Em implementações com listas em Python, a operação pop() utilizada numa Pilha tem complexidade O(1), enquanto o método pop(0), usado numa Fila, apresenta complexidade O(n), dado que exige o deslocamento de todos os elementos subsequentes da lista. Nos testes realizados, verificou-se que a Pilha apresentou um tempo de armazenamento ligeiramente inferior (0.000118 segundos) quando comparada com a Fila (0.000148 segundos). Para além disso, a sua implementação é igualmente simples, recorrendo apenas aos métodos append() e pop().

Contudo, estas vantagens de ordem técnica não compensam as limitações funcionais da Pilha no contexto específico deste projeto. Os dados de assiduidade dos alunos precisam de ser processados na mesma ordem cronológica em que foram registados, a fim de garantir uma análise fiável da evolução da presença ao longo do tempo. Como a Pilha segue o modelo LIFO, esta estrutura inverte a sequência dos dados, o que significa que os registos mais recentes seriam analisados primeiro. Essa inversão compromete a integridade da análise temporal, dificultando a identificação de padrões e a compreensão da progressão da assiduidade ao longo de diferentes momentos.

A Fila, por sua vez, ao operar segundo o modelo FIFO (First In, First Out), preserva naturalmente a ordem de inserção dos registos, o que a torna muito mais adequada do ponto de vista lógico e analítico. Assim, embora a Pilha possa ser tecnicamente mais eficiente em determinadas operações, ela revela-se conceptualmente inadequada para o armazenamento de

dados de assiduidade, ao passo que a Fila, mesmo com um desempenho ligeiramente inferior, garante a coerência necessária à análise cronológica e cumpre integralmente os objetivos do projeto.

---

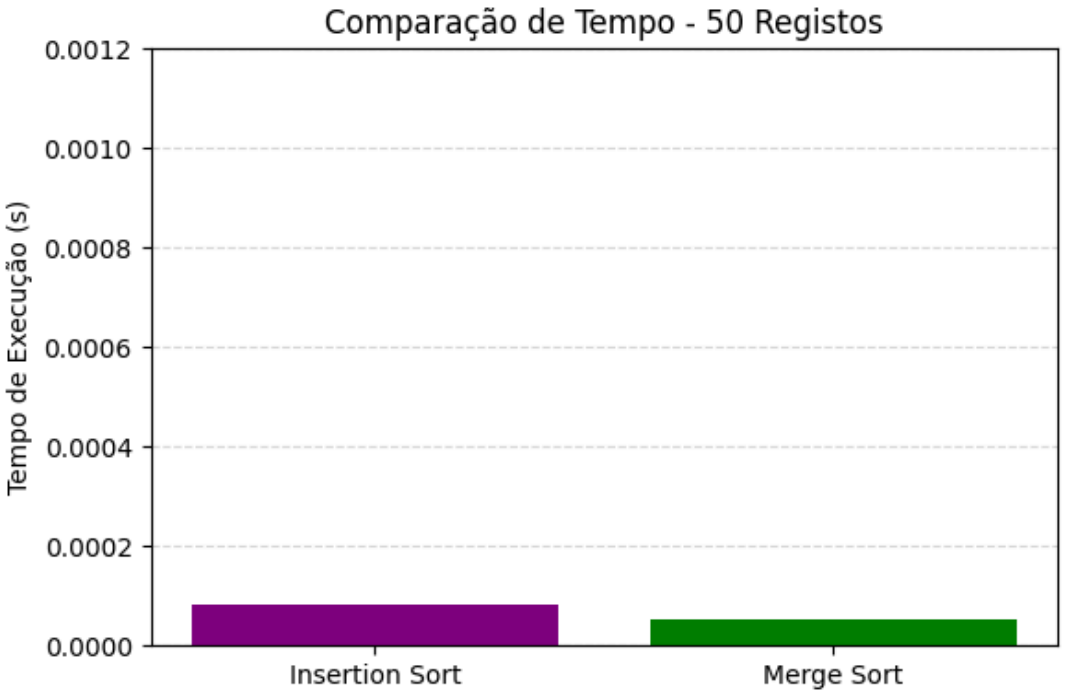
6) "**Quem são os alunos que contam com o maior e o menor número de presenças?"**

Com base nos dados analisados, verificou-se que a aluna com maior número de presenças foi Sofia Cardoso, com um total de 64 dias. Já o aluno com menor número de presenças foi Lucas Lima, com 36 dias

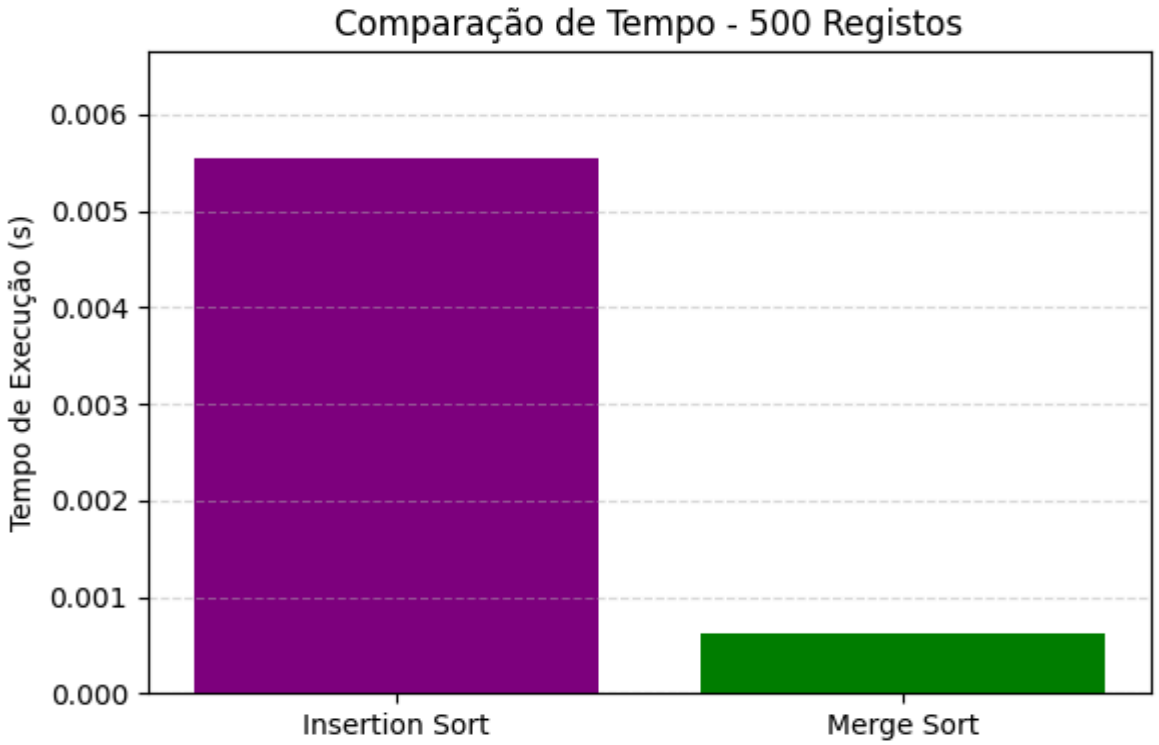
Anexos

Anexo A - Output do script após execução dos algoritmos de ordenação para 50 registos (com a limpeza dos dados divergentes)

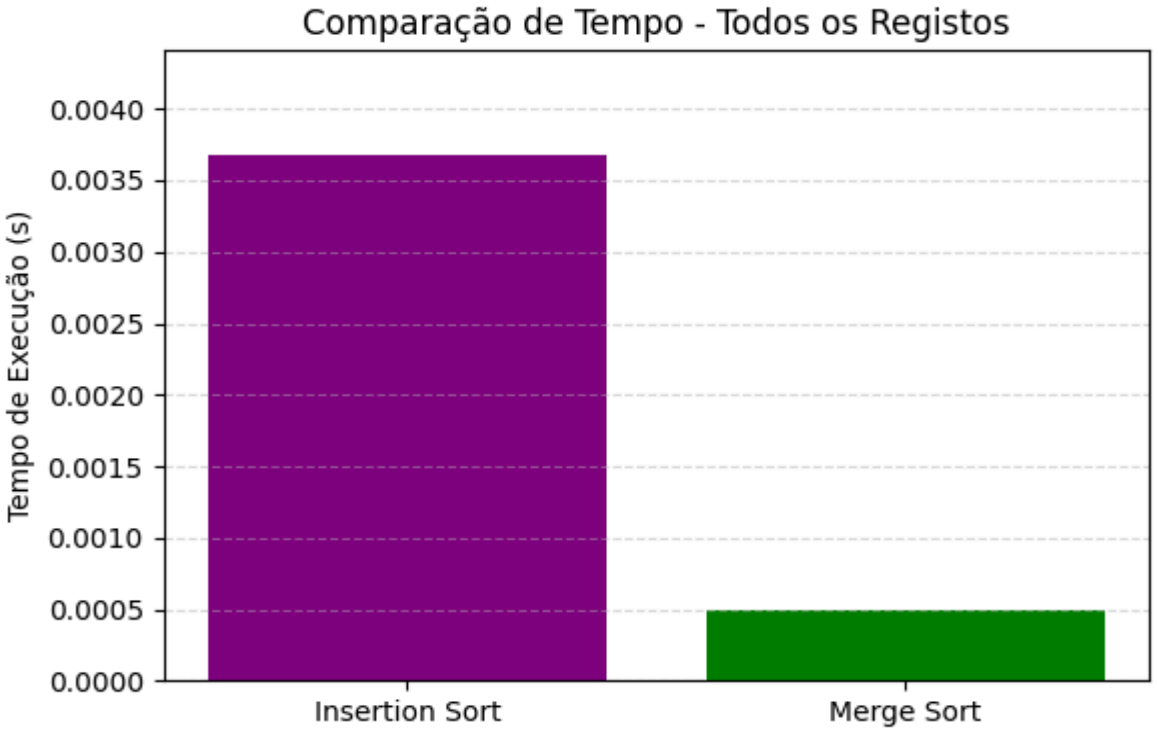
```
"C:\Users\fabio\PycharmProjects\EDA aula_1\.venv\Scripts\python.exe" "C:\Users\fabio\PycharmProjects\EDA aula_1\mmm.py"
Aluno com mais presenças: Daniel Ferreira (26 dias)
Aluno com menos presenças: Nicolau Barbosa (2 dias)

Tempos finais (em segundos) dos algoritmos:
50 Registos → Insertion: 0.000081s | Merge: 0.000054s
500 Registos → Insertion: 0.003997s | Merge: 0.000500s
Todos os Registos → Insertion: 0.003672s | Merge: 0.000492s

Tempo para armazenar na Fila: 0.000026s
Tempo para armazenar na Pilha: 0.000022s

Process finished with exit code 0
```



Anexo B - Output do script após execução dos algoritmos de ordenação para 500 registos (com a limpeza dos dados divergentes)

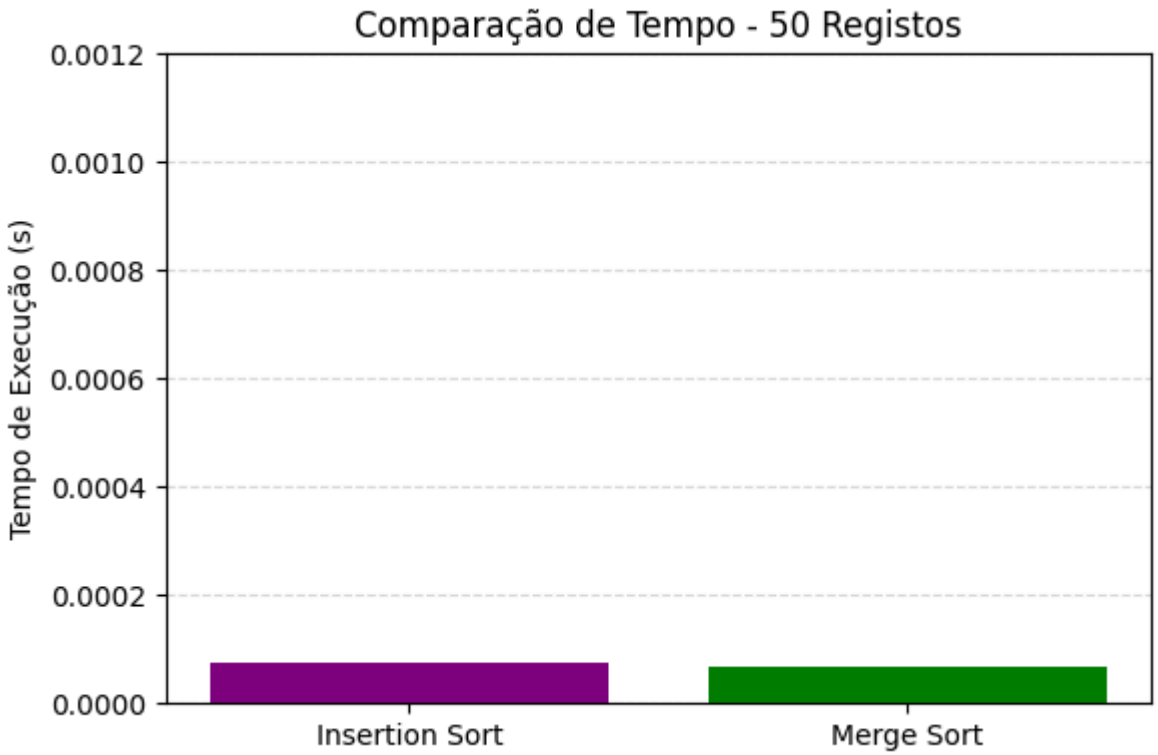


Anexo C - Output do script após execução dos algoritmos de ordenação para todos os registos (com a limpeza dos dados divergentes)

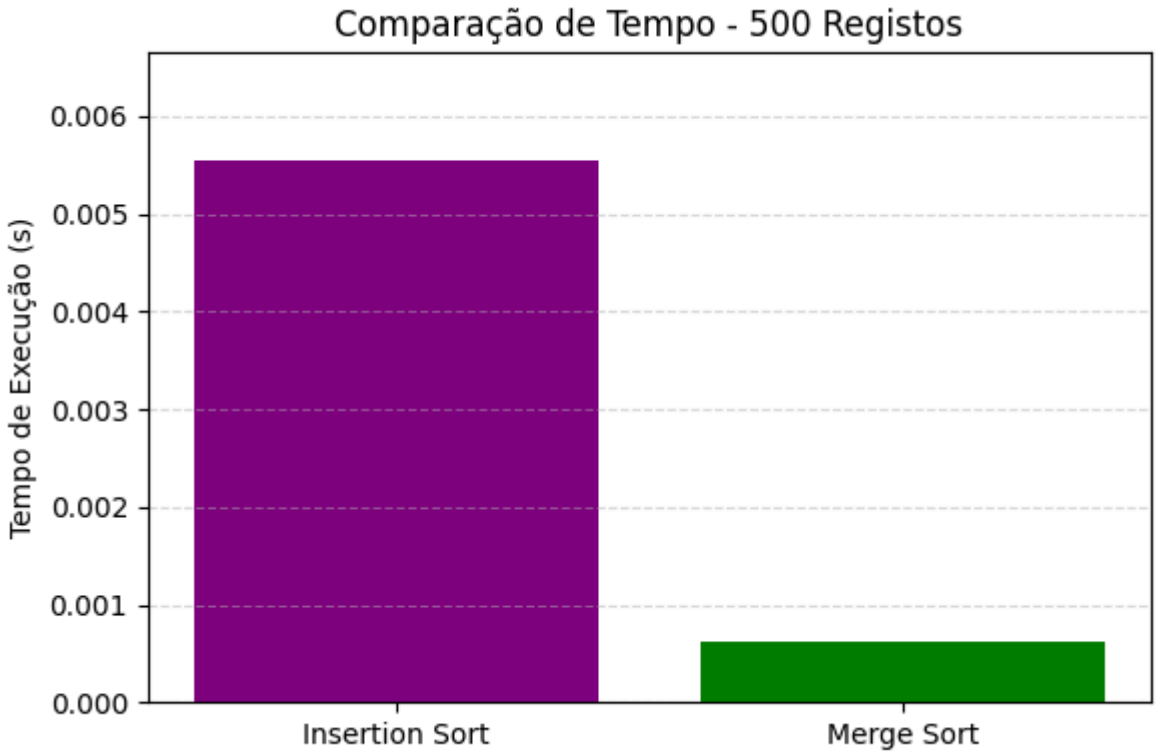
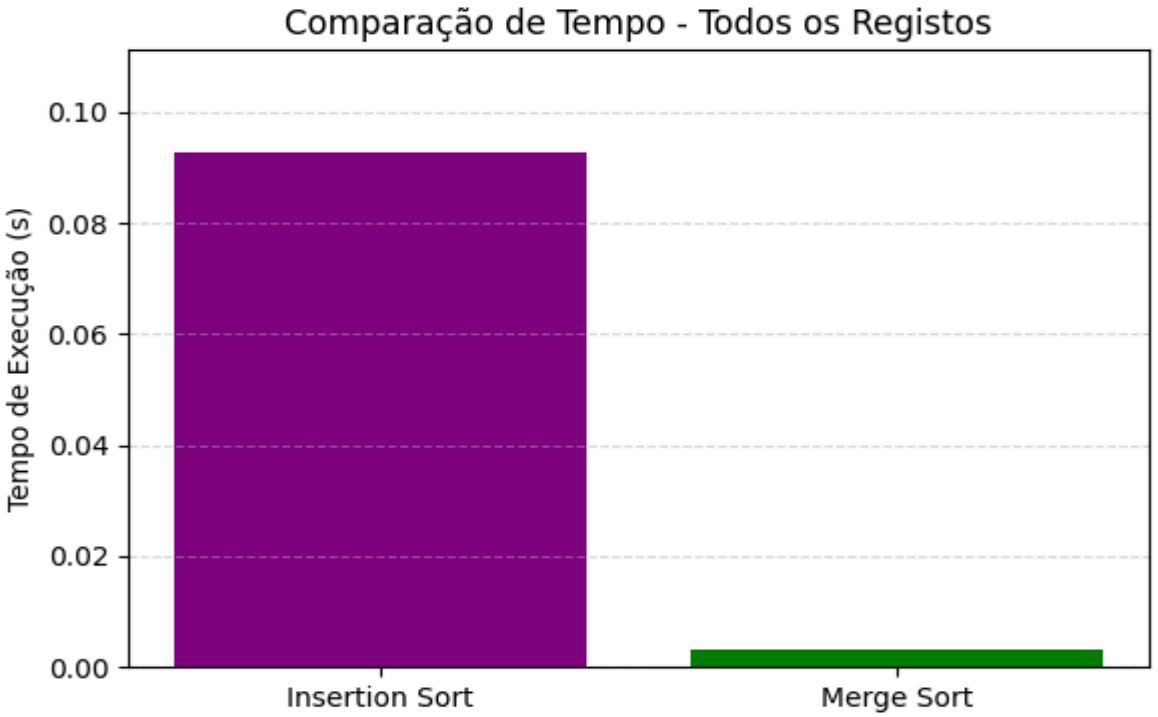


Anexo D - Output do script após execução dos algoritmos de ordenação para 50 registos (sem a limpeza dos dados divergentes)

```
"C:\Users\fabio\PycharmProjects\EDA aula_1\.venv\Scripts\python.exe" "C:\Users\fabio\PycharmProjects\EDA aula_1\mmm.py"
Aluno com mais presenças: Sofia Cardoso (64 dias)
Aluno com menos presenças: Lucas Lima (36 dias)

Tempos finais (em segundos) dos algoritmos:
50 Registos → Insertion: 0.000077s | Merge: 0.000053s
500 Registos → Insertion: 0.005480s | Merge: 0.000614s
Todos os Registos → Insertion: 0.090026s | Merge: 0.002951s

Tempo para armazenar na Fila: 0.000148s
Tempo para armazenar na Pilha: 0.000118s

Process finished with exit code 0
```



Anexo F - Output do script após execução dos algoritmos de ordenação para 500 registos (sem a limpeza dos dados divergentes)



Anexo G - Output do script após execução dos algoritmos de ordenação para todos os registos (sem a limpeza dos dados divergentes)